

Simulation 3D Fireworks Based on Particle System



Department of Computer Science

Course: Computer Graphics

Professor: Dr. Ian Stavness

Student: Arman Heydari

Date: December 2023

Algorithm Outline

The objective of this project is to implement an OpenGL application that simulates a firework using a lot of particles. If I want to explain it with a big picture of the approach, there are steps that should work if we want to shape an object with particles:

- Load an object shape. Here is a .obj file, which is the target shape of ours.
- Cluster the points of that object with K-means approach. I saved the cluster centers so we can use them and match the particles of our firework to this cluster. (Jesse Hall, John C. Hart, 2004)
- I used the mesh instancing approach (opengl-tutorial.org). As it takes a lot of resources if you want to use the cpu and update each of the particles at any frame. the instancing approach issues a single draw call that renders multiple instances of a mesh at once.
- Initialize a lot of particles with a position and speed and direction to fall down just like a firework explosion. We should specify a cluster for each of the particles to reach there, and we should compute the distance of that particle with the target position and also set a time to reach the target, to find the initial speed. Also, we should determine a lifetime for them because fireworks always fade after seconds.
- At each time frame we should do all these: Update the speed on each particle, change the particle position, reduce the lifetime of the particle, and delete the particle if the lifetime is over.
- Give the user the ability to change the zoom and place of the firework with arrow keys, and rotate the camera so you can fill the 3D environment.

Algorithm Details

Here, I will explain the important parts of the approach.

Clustering: We use the K-means approach to cluster the target object's vertices.

1. The process starts by randomly assigning random values as cluster centers. I've generated a random number between the maximum and minimum of the x, y, z of vertices of the object for this part. So we just need to set the number of clusters (It depends on the complexity of object that what number is fine and I set that with try and error), then generate $3 \times \text{CLUSTER_NO}$ random numbers that each bounds between the target objects coordinates.
2. Evaluating each point, assigning it to the closest cluster. The definition of "closest" is that the Euclidean distance between a data point and a cluster center is shorter than the distances to the other cluster centers.
3. Update all the cluster centers as the mean of all the points that are assigned to this cluster.
4. We should continue the second and third steps until convergence. There are different ways to define convergence. It can be a maximum iteration number that we define, or it can be until no

point changes between clusters or it can be a bound for error. I chose the second option just like (Hanli Zhao, Ran Fan, Charlie C. L. Wang, Xiaogang Jin, Yuwei Meng, 2009)

Initializing Particles:

I put their position at (0, 20, 0), so the explosion happens approximately at the top of the window. Before the explosion, when we created the particle array, all of the lifetime attributes were -1 and other attributes of the particle were zero. After the explosion, we set the lifetime to 5, because every particle should be deleted after 5 seconds. But, we set the clusterTime attribute to 4, because it should take 4 seconds for each particle to reach the destination cluster. But how should it know the destination? We specify the target cluster center for each particle, and we do it equally so every cluster has newParticles/CLUSTER_NO members (particles). Now we can compute the distance that each particle should take to the target position and save that too. Then, we compute the C value with respect to formulas for x, for y, and for z and save it for each particle. Also, we set the mass of all of them, their size, and their color equally. In this phase, the speed is (0,0,0) for them.

Updating Particles:

If a particle is still alive, which means the lifetime>0, we should update some of its attributes at every time frame. The first thing is the time variable that should be updated, and the delta time should be computed because we need it for updating the position. We update the position easily by multiplying the delta time in the speed vector. The distance to the camera also updates because the position is changed. The most important update is the velocity in the x,y,z directions which is done by functions using [formulas](#). We should set all the changes in the vertex buffer and color buffer so the rendering is done perfectly on the GPU.

Mathematical Details and Notations

M: initial mass of a particle, g: gravity acceleration, T: lifetime of the particle, t: time spend after explosion, k: air friction coefficient, S: distance of the point at t=0 with the cluster

In the Y axis, These are the formulas (Hanli Zhao, Ran Fan, Charlie C. L. Wang, Xiaogang Jin, Yuwei Meng, 2009):

$$C = \left(S - \frac{Mg(2TT_0 - T_0^2)}{2(kT - M)} \right) \frac{\frac{kT}{M} + 1}{T^{\frac{kT}{M}+1} - (T - T_0)^{\frac{kT}{M}+1}}$$

C: a constant that we need to calculate for finding the velocity.

$$v = C(T - t)^{\frac{kT}{M}} + \frac{Mg}{kT - M}(T - t)$$

So, this is how we can have the velocity of each particle at time t . the formulas are a little different in x and z axis as there is no gravity (Hanli Zhao, Ran Fan, Charlie C. L. Wang, Xiaogang Jin, Yuwei Meng, 2009):

$$C = S \frac{\left(\frac{kT}{M} + 1\right)}{T^{\frac{kT}{M}+1} - (T - T_0)^{\frac{kT}{M}+1}}$$

$$v = C(T - t)^{\frac{kT}{M}}$$

Implementation Description and Notations

The project provided [here](#) publicly. The logic of the code is in the following files.

textures.cpp: I use this file to apply clustering algorithm.

- CLUSTER_NO is the number of clusters
- float distance(const glm::vec3& a, const glm::vec3& b): This function calculates the Euclidean distance between two points a and b , which are 3D vectors.
- float generateRandomFloat(float x , float y): This function generates a random float between two numbers x and y .
- initializeClusterCenters: This function initializes the cluster centers. It takes a vector of 3D vectors to find the boundaries just like I explained.
- assignToCluster: This function assigns each point to the nearest cluster center. It takes a 3D vector point and a vector of 3D vectors clusterCenters as input.
- updateCenter: This function updates the cluster center by taking the average of all points in the cluster. It takes a vector of 3D vectors cluster as input.
- kMeans: This is the main K-means clustering function. It takes a vector of 3D vectors data as input, that here it's the target object's points.

controls.cpp: This file helps to move the camera closer or further of the firework, or move it left or right. The code is just like (opengl-tutorial.org), and I just removed the mouse controls.

Shaders: The shaders are also similar to (opengl-tutorial.org). VertexShader takes a square, faces it toward the camera, places it in the scene according to the particle's size and position, and gives it its color. By doing this for every particle in the particle system, I can render a lot of particles effectively. The UV coordinates can be used to map textures onto the particles. In real-time graphics, this method is frequently used. FragmentShader just applies the texture to the particle's color. I'm just using a same color for all of the particles so it's not complicated for this project.

teapot.cpp: This is where I create, initialize, update, and delete all the particles using buffers. I'm not going to explain this file line by line, but the functions and particle attributes that matter in the instancing method are:

- Particle struct: This is the definition of each particle and it has many attributes. pos, speed: The position and speed of the particle, represented as 3D vectors. r, g, b, a: The color of the particle. The operator function used to sort particles in reverse order based on their distance to the camera. CY, CXZ: constants used in calculating speed on the direction that their name shows. life: The remaining life of the particle. If it's less than 0, the particle is considered dead, this is "T" in the formulas. clusterTime: The amount of time it takes for the particle to reach the target cluster. This is less than life, and it's "T0" in the formulas. clusterIndex: Represents the target cluster for the particle, and it's a particular index of the clusterCenters array for a particle. clusterDistance: The euclidean distance to the target cluster, that's "S" in the formulas. initialMass: The mass of the particle at the beginning, that's "M" in the formulas.
- clusterCenters variable: an array of all the centers that calculated in the texture.cpp file.
- maxParticles variable: shows the maximum number of particles.
- gravityCoefficient variable: It's "g" in the formulas, airCoefficient variable: the air friction, It's "k" in the formulas.
- eps variable: It's epsilon that prevents division by zero.
- time variable: this is the time that spends from the explosion. It's "t" in the formulas.
- CalculateCXZCoefficient, CalculateCYCoefficient, CalculateXZVelocity, CalculateYVelocity functions: They are implemented just like the formulas using the particle attributes and the time variable. All of them just return a float.
- main function: after some basic common OpenGL codes, 3 buffers defined for One buffer for the vertices of the mesh, One buffer for the particles' centers, One buffer for the particles' colors (opengl-tutorial.org). Then the particles initialization step is done just like the explanation in [algorithm details](#), and in the do-while that ends just with closing the window, the updating particles is done.

References

Hanli Zhao, Ran Fan, Charlie C. L. Wang, Xiaogang Jin, Yuwei Meng. (2009). Fireworks Controller. *Wiley*.

Jesse Hall, John C. Hart. (2004). GPU Acceleration of Iterative Clustering.

opengl-tutorial.org. (n.d.). *Tutorial 18: Particles and object instancing*.